第六节：BaseApp 基类全解析

1. 课程目标

在开发 App 之前，我们必须理解所有 App 的“父亲”——`BaseApp`。它帮我们处理了大部分重复工作。本节课我们将详细的学习 `base_app.py` 的实现。

2. 核心源码分析

文件路径: `src/canmv/port/builtin_py/ybMain/base_app.py`

2.1 初始化 (`__init__`)

```
class BaseApp:
    def __init__(self, app_manager, name, icon=None):
        self.app_manager = app_manager # 保存appmanager的引用，方便以后申请返回桌面
        self.name = name               # App 名字
        self.icon = icon               # App 图标对象
        self.screen = None             # App 的主屏幕 (稍后创建)

        # 获取屏幕分辨率
        self.disp = lv.disp_get_default()
        self.disp_w = self.disp.get_hor_res()
        self.disp_h = self.disp.get_ver_res()
```

注意: 在 `__init__` 阶段，我们通常**不创建 UI**。因为系统启动时会一次性初始化所有 App，如果每个 App 都在这里创建几百个控件，内存瞬间就爆了（可能）。

2.2 创建桌面图标 (`create_icon`)

这个方法由 `AppManager` 调用。

```
    def create_icon(self, parent):
        self.icon_btn = lv.btn(parent) # 创建一个按钮

        # 设置图标文本
        icon_label = lv.label(self.icon_btn)
        icon_label.set_text(self.name)

        # 绑定点击事件
        self.icon_btn.add_event(self.on_icon_clicked, lv.EVENT.CLICKED, None)
        return self.icon_btn
```

2.3 启动与动画 (`launch_with_animation`)

这是 `BaseApp` 最精彩的部分。当你点击图标时，它不会生硬地跳转，而是有一个从图标位置放大到全屏的动画。

```
    def launch_with_animation(self):
        # 1. 创建新屏幕 (这时候才真正创建 App 的 UI 容器)
        self.screen = lv.obj()
```

```
        # 2. 获取图标当前的位置
        icon_coords = lv.area_t()
        self.icon_btn.get_coords(icon_coords)

        # 3. 设置初始状态：大小等于图标，位置等于图标
        self.screen.set_size(icon_w, icon_h)
        self.screen.set_pos(icon_x, icon_y)
        self.screen.set_style_radius(start_radius, 0) # 圆形

        # 4. 创建动画
        anim = lv.anim_t()
        # ... 设置动画从 icon_size 变大到 display_size ...

        # 5. 动画结束后，调用 initialize()
        # 这意味着子类必须实现 initialize() 方法
```

3. 开发App需要注意什么?

继承 `BaseApp` 后，你只需要关心两件事：

1. **准备数据**: 在 `__init__` 里准备好名字和图标。
2. **构建界面**: 重写 `initialize(self)` 方法，在 `self.screen` 上画出你的应用界面。